



Concurrency Control



Content

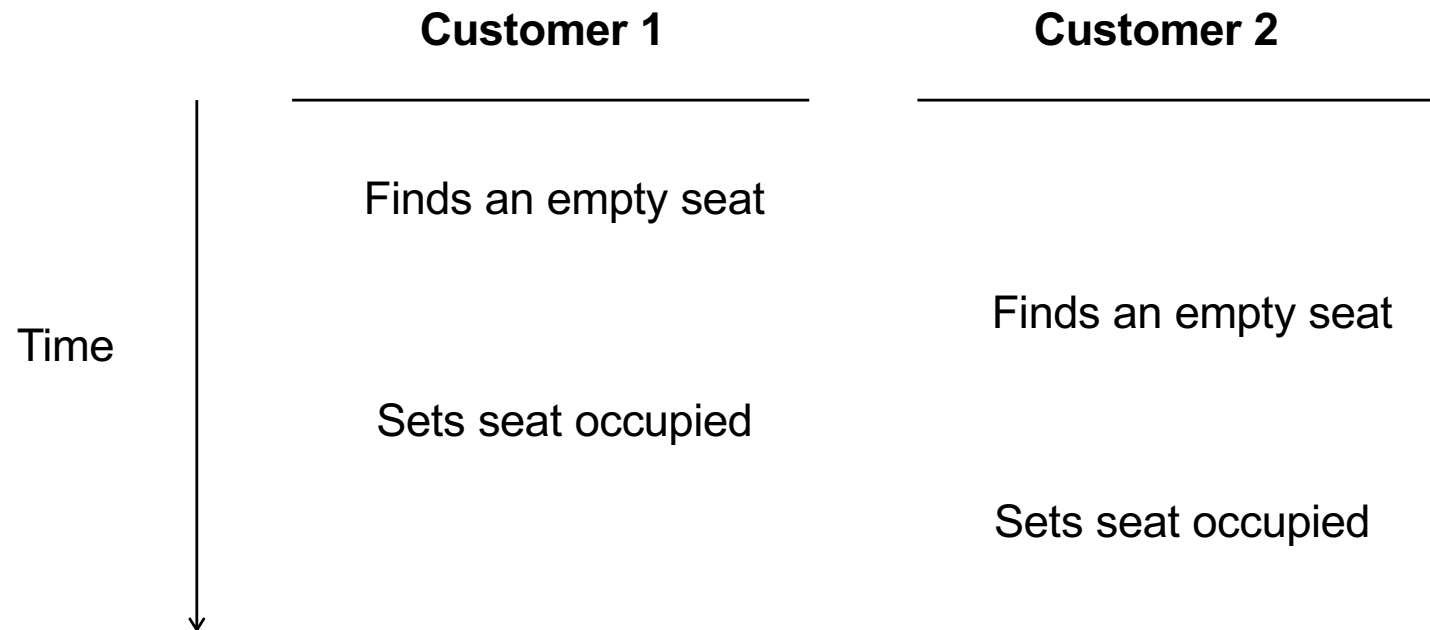
- **Introduction**
- Transaction
- Concurrency control
 - Scheduler
 - Schedule
 - Serial schedule
 - Serializable schedule

Example 1

- Airline reservation system
 - Hundreds of operations per second may be performed on the database
 - Operations initiate at any sites
 - Automatic teller machines
 - Machines on the desks of travel agents
 - Airline employees
 - Airline customers
 - Two operations could affect the same flight and overlap in time

Example 1

- Airline reservation system



Two customers are trying to book the same seat simultaneously ???

Discussion

- What could go wrong if DBMS were completely unconstrained?
 - Two operations could each be performed correctly
 - But the global result will be not correct
 - Both customers believe they have been granted the seat
- DBMS is a multi-user system
 - Many operations access the same data unit at the same time
 - Many operations are performed concurrently
- To solve the problem of Example 1
 - Mechanism: ***serialize*** the execution of the two operations

Serilizability

- An execution of operations on the same DB is ***serial***
 - If one operation executes completely before any other operation begins
- The execution is ***serializable***
 - If operations *behave as if they were run serially*, even though their executions may overlap in time

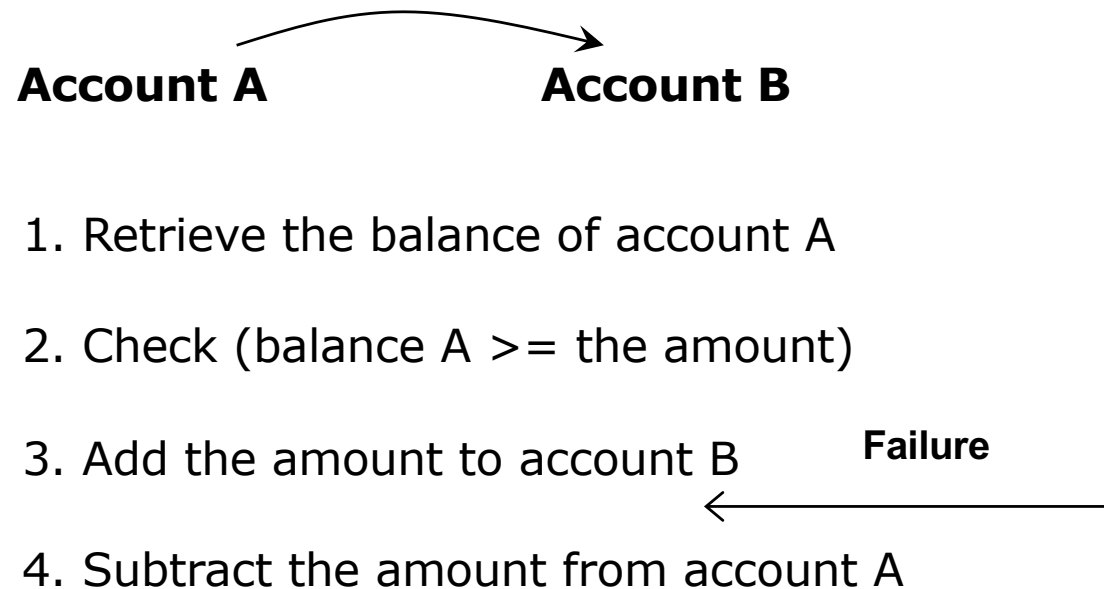
Discussion

■ Example 1

- If two operations are run serially
 - One customer's request occurs first
 - This customer sees an empty seat and books it
 - The other customer's request then begins and sees that the seat is already occupied
- The important thing to
 - The customer: who gets the seat
 - The database: a seat is assigned only once

Example 2

- Banking system
 - Transfer money from A to B



The bank has in effect given away the amount of money that was transferred ???

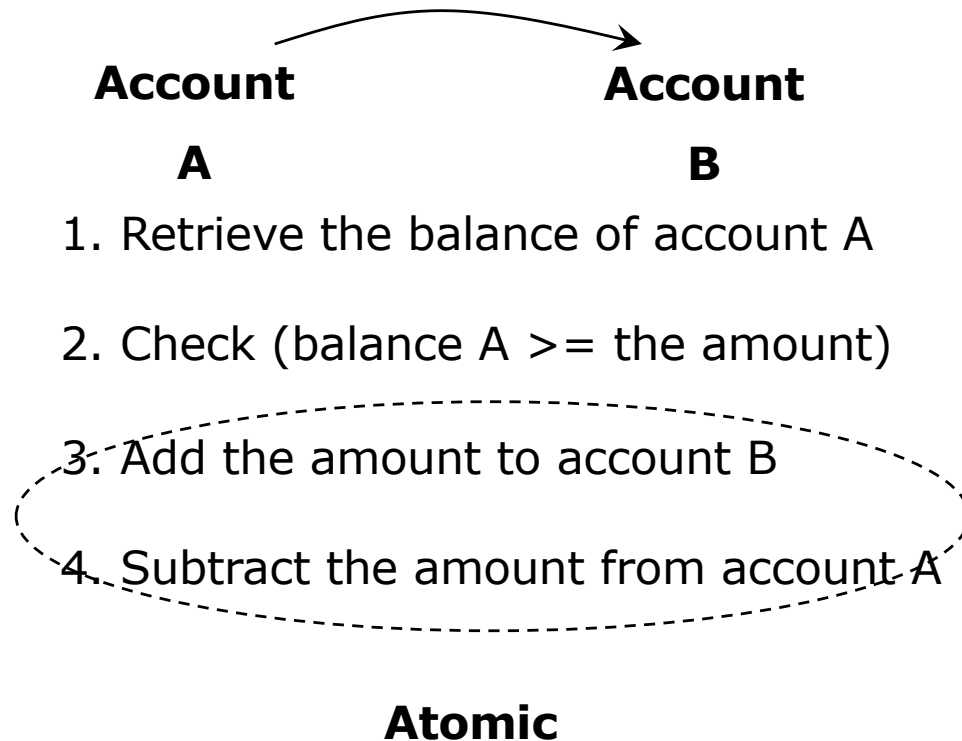
Discussion

■ Example 2

- Solution: the 3rd and 4th operations could not be separated
 - Or both operations are performed and have to be completed
 - Or none are performed
 - * If the 3rd operation is done and the 4th operation is failure
 - * Then the 3rd operation has to be canceled
- The combination of the 3rd and 4th operations need to be done ***atomically***

Atomicity

- Either all operations are done or neither is done
 - All or nothing



Discussion

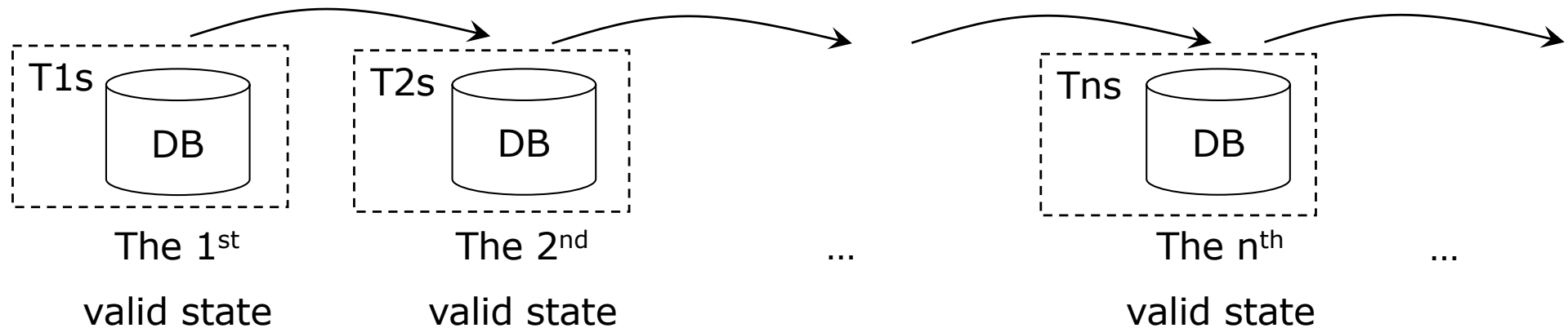
- How to solve the problems of ***serialization*** and ***atomicity***?
- The solution
 - Group operations into ***transactions***

Content

- Introduction
- **Transaction**
- Concurrency control

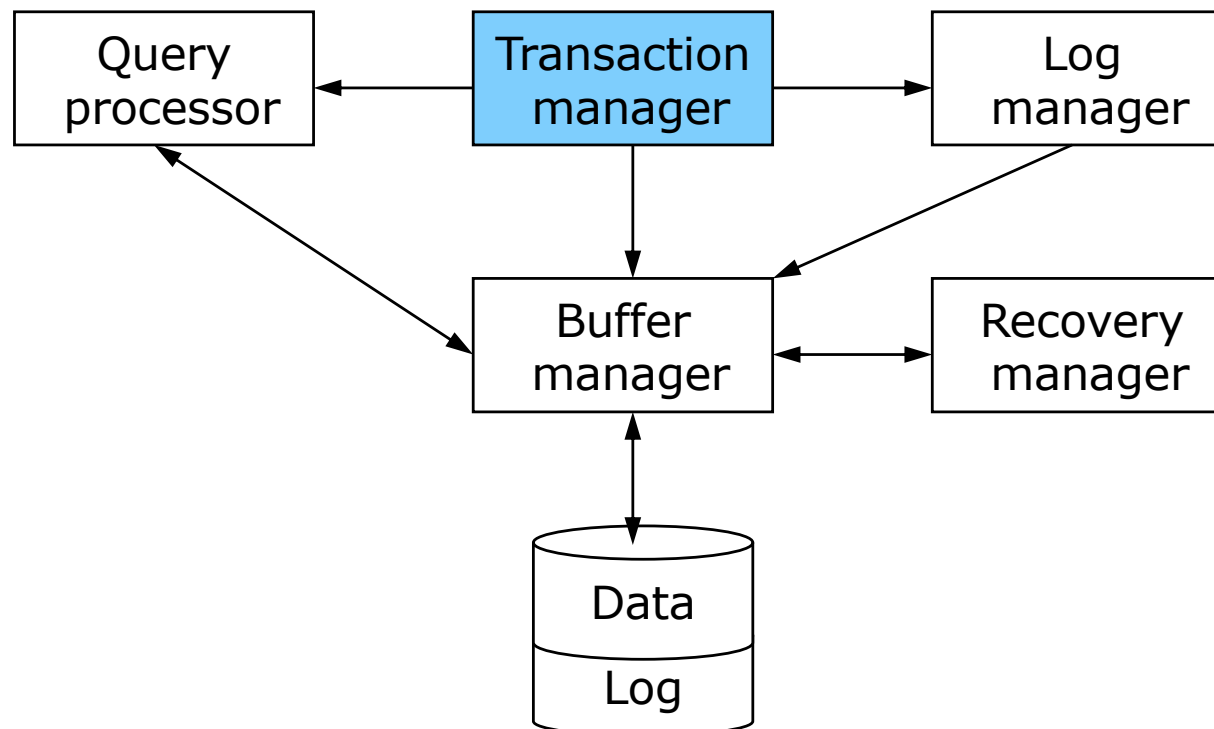
Transaction

- A collection of operations on the database that must be executed atomically, that is all operations are performed or none are



Transaction management

- A subsystem performs functions
 - Issue signals to log manager
 - Assure that concurrently executing transactions do not interfere with each other

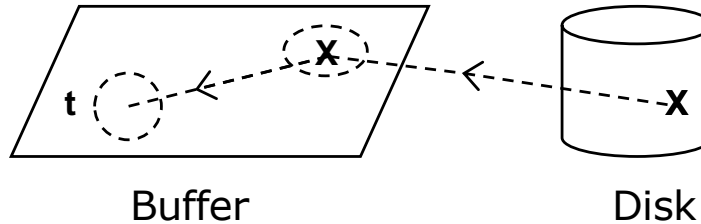


Transaction operation

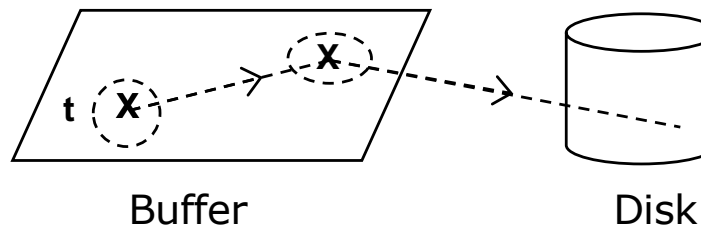
- Suppose that a database concludes many data units
- One data unit (element)
 - Has a value
 - Is accessed and modified by transactions
 - Can be
 - Relations
 - Disk blocks (pages)
 - Tuples

Transaction operation

- Input(X)
- Read(X,t)



- Write(X,t)
- Output(X)



- Buffer manager
 - Input & Output
- Transaction
 - Read & Write

Example

- Suppose that a database
 - Concludes 2 data units A and B
 - Has a constraint: in all cases, A is always equal B
- Transaction T wants to
 - $A = A * 2$
 - $B = B * 2$
- T is represented
 - Read(A,t); $t = t * 2$; Write(A,t)
 - Read(B,t); $t = t * 2$; Write(B,t)

Example

Operations	t	Mem A	Mem B	Disk A	Disk B
Read(A,t)	8	8		8	8
t:=t*2	16	8		8	8
Write(A,t)	16	16		8	8
Read(B,t)	8	16	8	8	8
t:=t*2	16	16	8	8	8
Write(B,t)	16	16	16	8	8
Output(A)	16	16	16	16	8
Output(B)	16	16	16	16	16

ACID properties

- **A**tomicity
- **C**onsistency
- **I**solation
- **D**urability

ACID properties

■ Atomicity

- Either all operations of the transaction are reflected properly in the database, or none are

■ Consistency

■ Isolation

■ Durability

```
T: Read(A, t) ;  
    t:=t-50 ;  
    Write(A, t) ;  
    Read(B, t) ;  
    t:=t+50 ;  
    Write(B, t) ;
```

- A=100, B=200 (A+B=300)
- After Write(A,t)
 - * A=50, B=200 (A+B=250): DB is in an inconsistent state
- After Write(B,t)
 - * A=50, B=250 (A+B=300): DB is in a consistent state
- If transaction T never started or was guaranteed to complete, inconsistent state would not be visible except during the execution

ACID properties

- **A**tomicity
- **C**onsistency
 - Execution of a transaction *in isolation* (that is with no other transaction executing concurrently) preserves the consistency of the database
- **I**solation
- **D**urability

```
T: Read (A, t) ;  
   t:=t-50 ;  
   Write (A, t) ;  
   Read (B, t) ;  
   t:=t+50 ;  
   Write (B, t) ;
```

- The sum of A and B is unchanged by T
- If the DB is consistent before an execution of T, the DB remains consistent after the execution of T

ACID properties

- **A**tomicity
- **C**onsistency
- **I**solation
 - Each transaction is unaware of other transactions executing concurrently in the system
- **D**urability

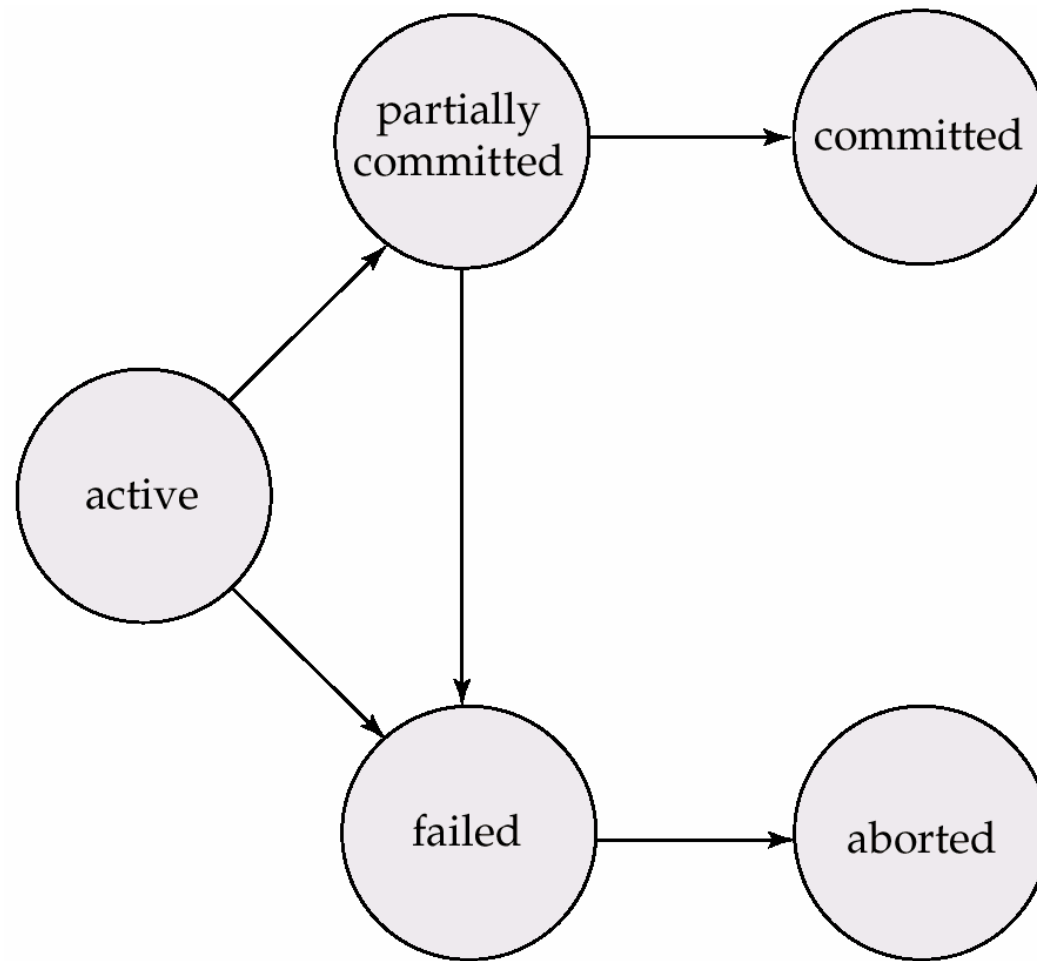
```
T: Read(A, t);  
    t:=t-50;  
    Write(A, t);  
T' → Read(B, t);  
      t:=t+50;  
      Write(B, t);
```

- If transaction T' wants to compute A+B during the execution of transaction T
- After the completion of T':
 - * $A+B=50+200=250$
- After the completion of T:
 - * $A+B=50+250=300$
- Concurrent execution of transactions in a state that is equivalent to a state that transactions executed one at a time in some order

ACID properties

- **A**tomicity
- **C**onsistency
- **I**solation
- **D**urability
 - After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures

Transaction state



Source: Silberschatz et al., 6E

Content

- Introduction
- Transaction
- **Concurrency control**
 - Scheduler
 - Schedule
 - Serial schedule
 - Serializable schedule

Transaction processing

■ Serial

- At a certain time, a transaction can only start if the previous one has completed

■ Concurrency

- Transactions are performed simultaneously
- Transactions can access the same data

→ But DB state can become inconsistent, even when

- Transactions individually preserve correctness of the state
- There is no system failure

Why concurrency control?

- Utilize resources and throughput
 - Processing on disk and CPU can be performed in parallel
 - While a transaction performs I/O on the disk, others perform computations on the CPU
 - CPU and disk have less idle time
- Reduce the waiting time
 - A transactions with a short running time has to wait for transactions with a long running time
 - When transactions access different data, they should be performed concurrently

Concurrency control

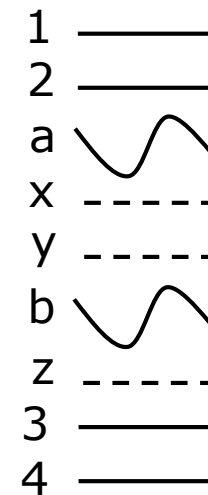
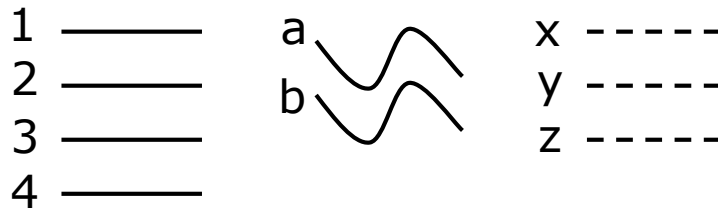
- The process of assuring that transactions preserve consistency when executing simultaneously
 - Transactions
 - Multiple actions within a transaction
 - The correctness of database state
 - Transform any consistent state to another consistent state

Scheduler

- The order in which the individual actions of different transactions occur needs to be regulated
- Controlling these actions is a task of the ***scheduler***
 - A component of DBMS takes charge of creating a ***schedule*** that performs concurrently executing transactions
- Role
 - Takes read/write requests from transactions
 - Either executes them in buffers or delays them

Schedule

- A ***schedule*** is a time-ordered sequence of the actions taken by one or more transactions
- Actions of a transaction in the schedule is the same order that they appear in the transaction itself



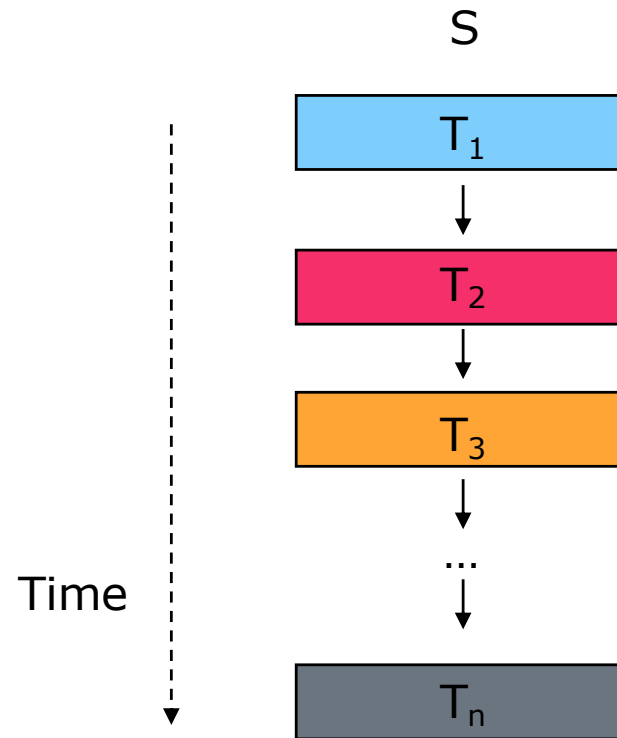
Example

T_1	T_2
Read(A,t)	Read(A,s)
$t := t + 100$	$s := s * 2$
Write(A,t)	Write(A,s)
Read(B,t)	Read(B,s)
$t := t + 100$	$s := s * 2$
Write(B,t)	Write(B,s)

- The consistency constraint: $A=B$
- Each transaction, run in isolation, will preserve consistency

Serial schedule

- A schedule is ***serial***
 - If its actions consist of all the actions of one transaction, then all the actions of another transaction, and so on, with no mixing of the actions



Serial schedule

S ₁	T ₁	T ₂	A	B	S ₂	T ₁	T ₂	A	B
			25	25				25	25
Read(A,t)						Read(A,s)			
t:=t+100						s:=s*2			
Write(A,t)			125			Write(A,s)	50		
Read(B,t)						Read(B,s)			
t:=t+100						s:=s*2			
Write(B,t)				125		Write(B,s)			50
		Read(A,s)			Read(A,t)				
		s:=s*2			t:=t+100				
		Write(A,s)	250		Write(A,t)			150	
		Read(B,s)			Read(B,t)				
		s:=s*2			t:=t+100				
		Write(B,s)		250	Write(B,t)				150

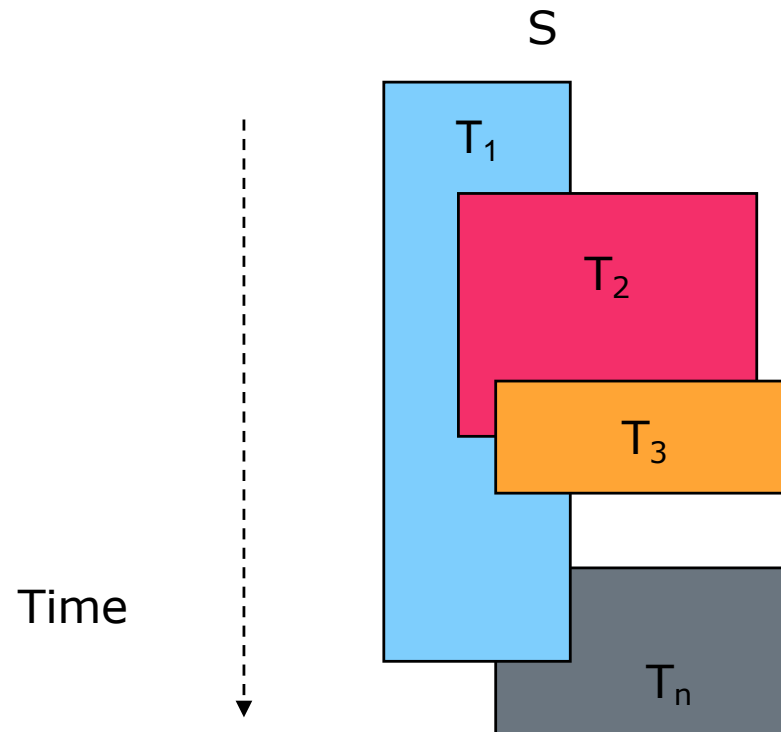
Are there any other schedules that also are guaranteed to preserve consistency?

Discussion

- Serial execution
 - Assure consistency
 - Slow
- Concurrently execution
 - Be able to violate consistency
 - Quick
- Compromise
 - Still assure consistency
 - Quick

Serializable schedule

- A schedule is ***serializable***
 - If its effect on the database state is the same as of some serial schedule, regardless of what the initial state of the database is



Example

S₃	T ₁	T ₂	A	B
			25	25
Read(A,t) t:=t+100 Write(A,t)			125	
		Read(A,s) s:=s*2 Write(A,s)	250	
Read(B,t) t:=t+100 Write(B,t)				125
		Read(B,s) s:=s*2 Write(B,s)		250

- Before S₃ performs
 - A=B=c
 - c: constant
- After S₃ completes
 - A=2*(c+100)
 - B=2*(c+100)
- DB state is consistency
- S₃ is serializable

Example

S₄	T ₁	T ₂	A	B
			25	25
Read(A,t) t:=t+100 Write(A,t)			125	
		Read(A,s) s:=s*2 Write(A,s)	250	
		Read(B,s) s:=s*2 Write(B,s)		50
Read(B,t) t:=t+100 Write(B,t)				150

- Before S₄ performs
 - A=B=c
 - c: constant
- After S₄ completes
 - A=2*(c+100)
 - B=(2*c)+100
- DB is in inconsistency state
- S₄ is not serializable

Example

S₅	T ₁	T ₂	A	B
			25	25
Read(A,t) t:=t+100 Write(A,t)			125	
		Read(A,s) s:=s* 1 Write(A,s)	125	
		Read(B,s) s:=s* 1 Write(B,s)		25
Read(B,t) t:=t+100 Write(B,t)				125

- After S₅ completes
 - A=B
- Final state is consistent
- S₅ is serializable
 - Result looks like serial schedules (T₁,T₂) or (T₂,T₁)

“Does this transaction multiply A by a constant other than 1?”

Discussion

- It is not realistic that
 - A scheduler has to concern itself with the details of computation undertaken by transactions
- Transactions often are written in
 - General purpose programming language
 - High level language statement
- However, scheduler
 - Get to see read/write requests from the transactions
 - Know what database elements each transaction reads, what elements it might change

Discussion

- How to simplify the job of scheduler?
- Assumption
 - If there is something that transaction T could have done to database element A that will make the database state inconsistent
 - Then T will do that
- We accept the computations performed by a transaction can be arbitrary
 - Do not need to consider the details of local computation actions
 - Only the reads/writes performed by transactions

Notation

- Read and write actions
 - $r_i(X)$ or $w_i(X)$
 - Transaction T_i reads or writes element X
- Transaction T_i
 - A sequence of actions with subscript i
- Schedule S of a set of transaction $\mathcal{T}$
 - An interleaving of the actions of transactions T_i in $\mathcal{T}$ such that
 - The actions of T_i appear in S in the same order that they appear in the definition of T_i itself

Example

- $T_1: r_1(A); w_1(A); r_1(B); w_1(B);$
- $T_2: r_2(A); w_2(A); r_2(B); w_2(B);$
- $S: r_1(A); w_1(A); r_2(A); w_2(A); r_1(B); w_1(B); r_2(B); w_2(B);$

Test for serializable

- To assure that a schedule is serializable
- Condition
 - A pair of consecutive actions in a schedule
 - If their order is interchanged
 - Then, the behavior of at least one of the transactions involved can change

T	T'	
Action 1 Action 2	Action 1' Action 2'	Conflict
Action 3 Action 4	Action 3' Action 4'	Not conflict

Not conflict

- Examine T_i and T_j , $i \neq j$
 - $r_i(X) ; r_j(Y)$
 - Is never a conflict, even if $X=Y$
 - Neither of these actions change the value of DB element
 - $r_i(X) ; w_j(Y)$
 - Is not a conflict when $X \neq Y$
 - T_j writes Y before T_i reads X , value of X is not changed
 - T_i reads X does not affect the value T_j writes for Y
 - $w_i(X) ; r_j(Y)$
 - Is not a conflict if $X \neq Y$
 - $w_i(X) ; w_j(Y)$
 - Is not a conflict as long as $X \neq Y$

Conflict

- The order of actions can not be swapped when
 - Two actions of the same transaction
 - $r_i(X) ; w_i(Y)$
 - Two actions of different transactions
 - Involve to the same DB element, and
 - At least one is a write

T_i	T_j
Write(X)	
	Write(X)

T_i	T_j
Read(X)	
	Write(X)

T_i	T_j
Write(X)	
	Read(X)

"no coincidences"

Example

S	T ₁	T ₂
	Read(A) Write(A)	
		Read(A) Write(A)
	Read(B) Write(B)	
		Read(B) Write(B)

T ₁	T ₂
Read(A) Write(A)	
	Read(A) Write(A)
Read(B) Write(B)	
	Read(B) Write(B)

S'	T ₁	T ₂
	Read(A) Write(A) Read(B) Write(B)	
		Read(A) Write(A)
		Read(B) Write(B)

Conflict equivalence

■ Definition

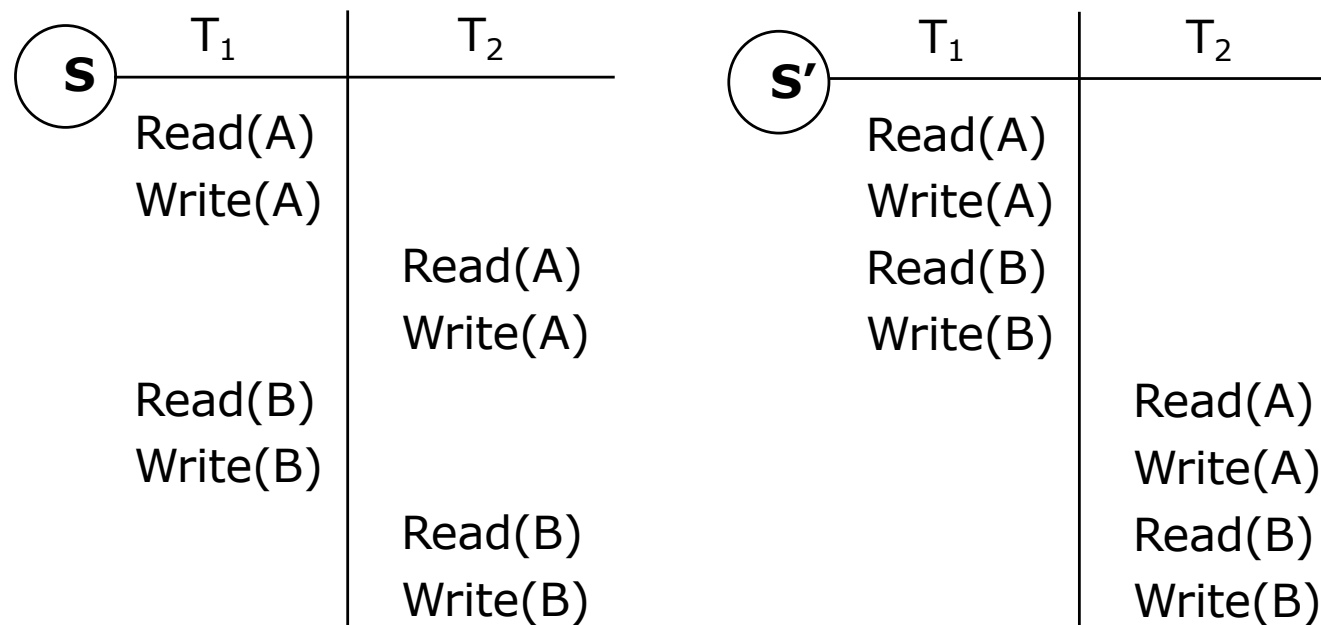
- S and S' are **conflict-equivalent**
 - If they can be turned one into the other by a sequence of non-conflicting swaps of adjacent actions
- S is **conflict-serializable**
 - If it is conflict-equivalent to a serial schedule

■ Note

- Conflict-serializability $\rightarrow$ serializability
- Conflict-serializability $\leftarrow$ serializability ???

Sufficient condition

- Conflict-serializability $\rightarrow$ serializability
 - A conflict serializable schedule is a serializable schedule



Necessary condition

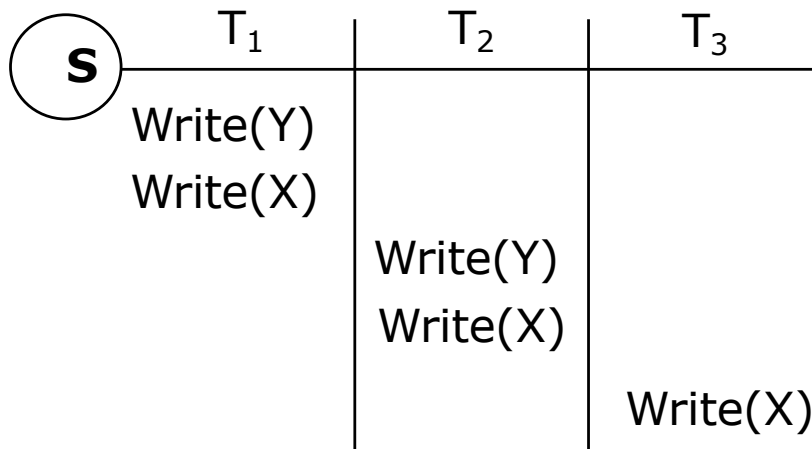
- Conflict-serializability $\leftarrow$ serializability ???

S₅	T ₁	T ₂	A	B
			25	25
Read(A,t) t:=t+100 Write(A,t)			125	
		Read(A,s) s:=s*1 Write(A,s)	125	
		Read(B,s) s:=s*1 Write(B,s)		25
Read(B,t) t:=t+100 Write(B,t)				125

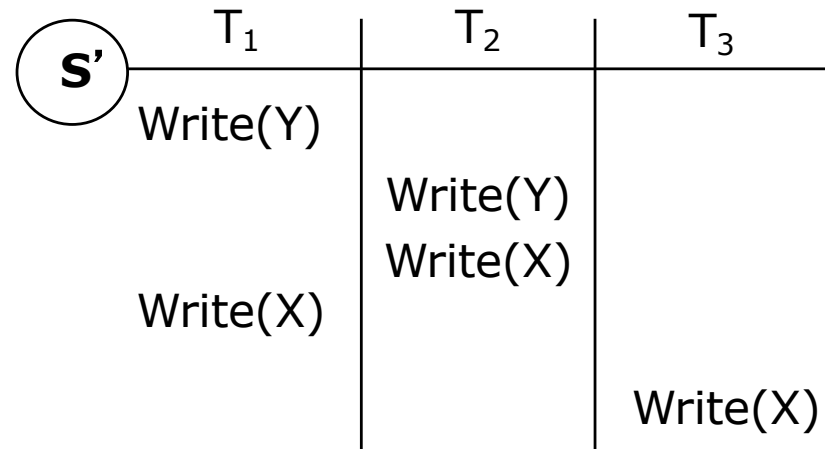
**Serializable
but not
conflict-serializable**

Necessary condition

- Conflict-serializability $\leftarrow$ serializability ???



Serial



Serialiable

but not

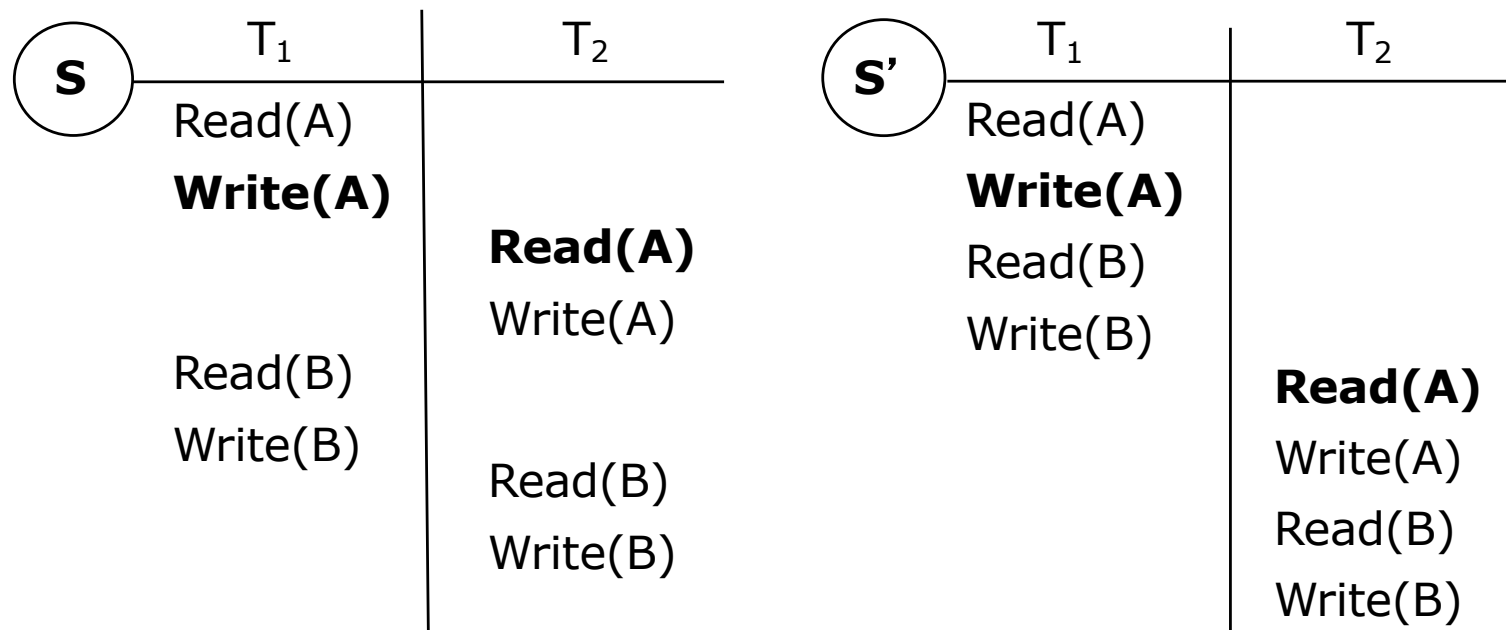
Conflict-serializable

Problem

- Examine a schedule S
- Decide whether or not S is conflict-serializable ???

Idea

- *Conflicting actions* in S
- **Transactions** performing these actions in S' must appear in the same order as the actions appear in S



Discussion

■ Relation

- The order of conflicting actions in S
- The order of transactions in S'

■ Constraint

- If they are not contradictory
 - We can find a conflict-equivalent serial schedule
- If they are contradictory
 - We know that no such serial schedule exists

How to test

- Give a schedule S involving T_1 and T_2
- Actions $A_1 \in T_1$
- Actions $A_2 \in T_2$
- **T_1 takes precedence over T_2 ($T_1 <_S T_2$) if**
 - A_1 is ahead of A_2 in S
 - Both A_1 and A_2 involve the same DB element
 - At least one of A_1 and A_2 is a write action

Discussion

- We cannot swap the order of A_1 and A_2
 - A_1 will appear before A_2 in any schedule that is conflict-equivalent to S
- If one of these schedules is a serial schedule, it must have T_1 before T_2

Precedence graph

- Give a schedule S involving $T_1, T_2, \dots, T_n$
- Precedence graph of S , $P(S)$
 - Nodes are transactions T_i of S
 - Arc from node i to node j if $T_i <_S T_j$
- Note
 - Any topological order of nodes is a conflict-equivalent serial order

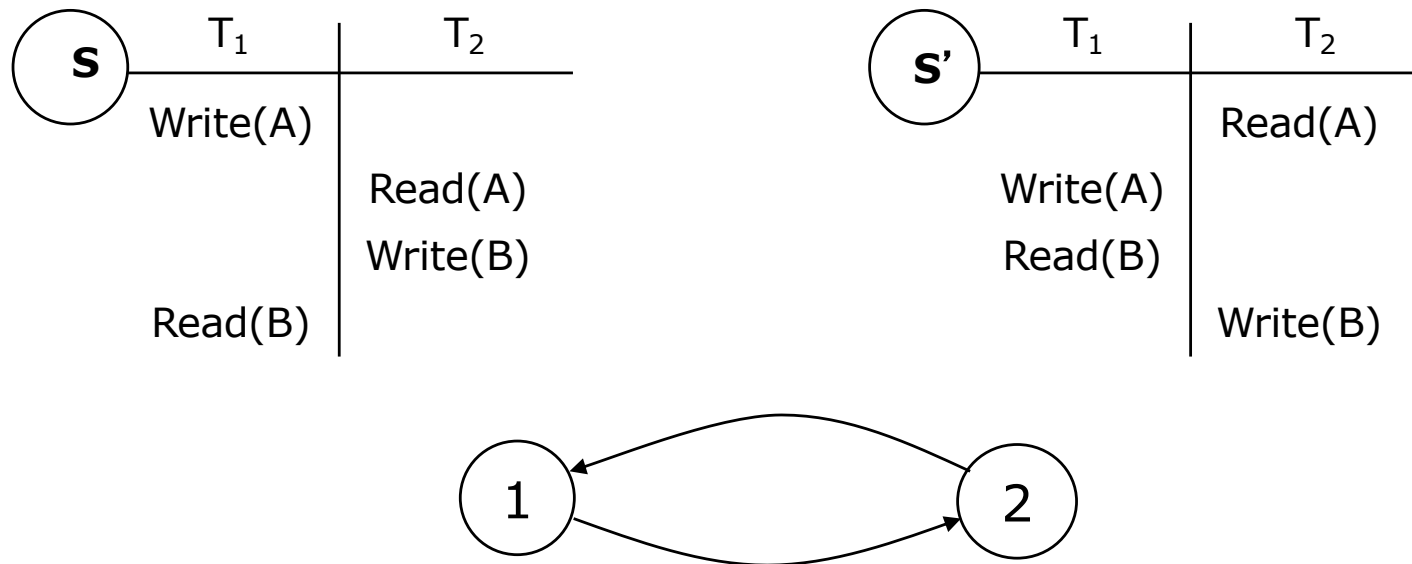
Lemma

- S and S' are conflict-equivalent
 $\Rightarrow P(S)=P(S')$
- Proof
 - Suppose that $P(S) \neq P(S')$
 - $\exists T_i$ such that $T_i <_S T_j$ exists in S, not in S'
 - $\Rightarrow S = \dots p_i(A) \dots q_j(A) \dots$
 $S' = \dots q_j(A) \dots p_i(A) \dots$
 - $\Rightarrow p_i(A)$ and $q_j(A)$ are conflict
 - $\Rightarrow S$ and S' are not conflict-equivalent

Note

- $P(S) = P(S')$
 $\Rightarrow S$ and S' are conflict-equivalent

- Examine 2 cases



Theorem

- If $P(S)$ is acyclic
- Then S is conflict-serializable

Proof

- $P(S)$ is acyclic $\Rightarrow S$ is conflict-serializable
 - Suppose that $P(S)$ has no cycles
 - Reorder S by using legal swaps of adjacent actions, as follows
 - Select one node that has no arcs into it (T_i)
 - Move T_i to the front of S
 $S = (\text{actions of } T_i)(\text{actions of the other } n-1 \text{ transactions})$
 - Consider the tail of S
 - * $P(\text{tail of } S)$ is the same as the $P(S)$, except that node T_i and any arcs out of this node are missing
 - * Since $P(S)$ is acyclic, $P(\text{tail of } S)$ is acyclic
 - Can reorder actions in the tail of S
 - Repeat until S turn into a serial schedule
 - S is conflict-serializable

Proof

■ $P(S)$ is acyclic $\Leftarrow$ S is conflict-serializable

- S is conflict-serializable
- $\exists S'$ is a serial schedule

$\Rightarrow S$ and S' are conflict-equivalent

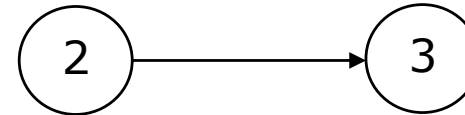
$\Rightarrow P(S) = P(S')$

- $P(S')$ is acyclic $\Rightarrow P(S)$ is acyclic

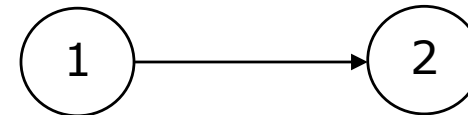
Example 1

S	T_1	T_2	T_3
		Read(A)	
Read(B)		Write(A)	
			Read(A)
Write(B)			Write(A)
		Read(B)	
		Write(B)	

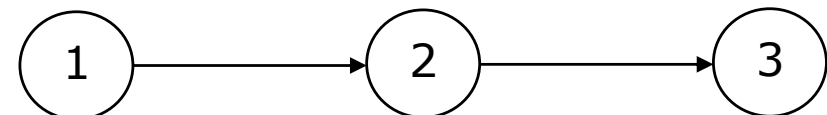
■ $T_2 <_S T_3$



■ $T_1 <_S T_2$



- $P(S)$ is acyclic
- S is conflict-serializable with the order of (T_1 , T_2 , T_3)



Example 1

S	T ₁	T ₂	T ₃
		Read(A)	
	Read(B)	Write(A)	
			Read(A)
	Write(B)		Write(A)
		Read(B)	
		Write(B)	

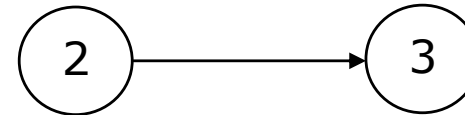
- S is conflict-serializable with the order of (T1, T2, T3)

Example 2

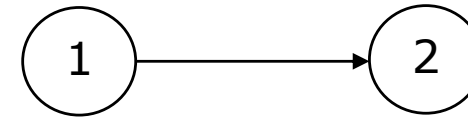
S	T_1	T_2	T_3
		Read(A)	
Read(B)		Write(A)	
		Read(B)	
Write(B)			Read(A)
			Write(A)
		Write(B)	

- $P(S)$ is cyclic
- S is not conflict-serializable

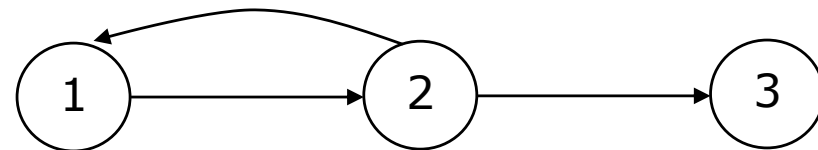
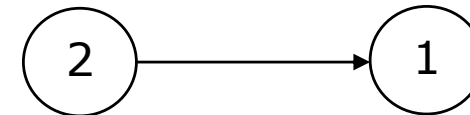
■ $T_2 <_S T_3$



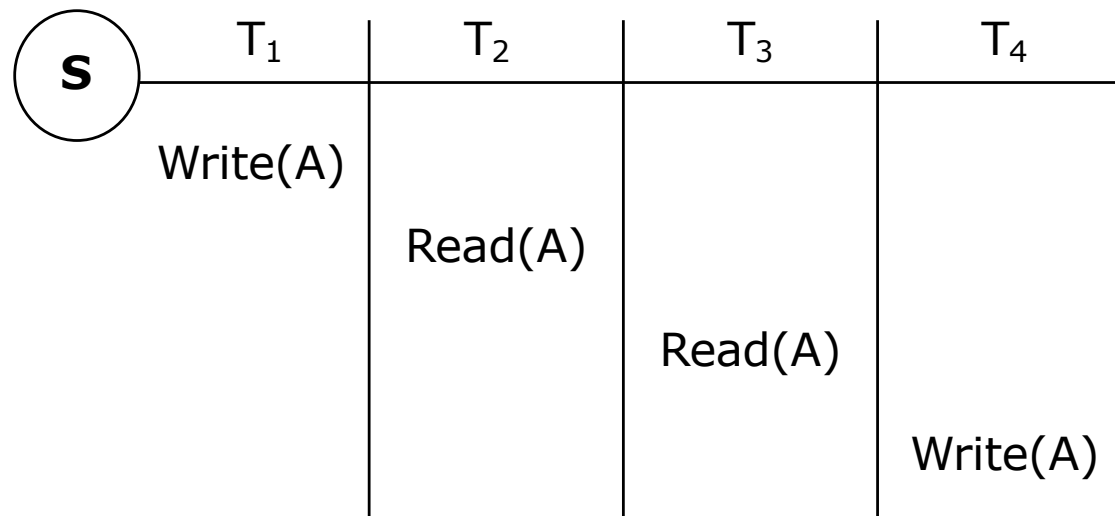
■ $T_1 <_S T_2$



■ $T_2 <_S T_1$



Exercise 1



- Draw P(S)
- Is S conflict-serializable?

Exercise 2

S	T ₁	T ₂	T ₃	T ₄
	Read(A) Write(B) Read(C)	Write(C) Write(A)	Write(A)	 Read(A) Write(D)

- Draw P(S)
- Is S conflict-serializable?

